MACHINE LEARNING MINI-PROJECT

BANKNOTE AUTHENTICATION USING LOGISTIC REGRESSION

A Binary Classification Study

Student details	Fill in before submission
Name	RAJASHREE KASHYAP
USN	4NI24CI083
Class / Semester	AIML-B 5TH SEM
Course	Machine Learning Laboratory
Department / Institution	Department of CSE – AI & ML, The National Institute of Engineering, Mysuru
Academic year	2026-27

Abstract

This mini-project develops a Logistic Regression classifier for the Banknote Authentication dataset from the UCI Machine Learning Repository. The dataset contains 1,372 records, four numerical input features—Variance, Skewness, Curtosis and Entropy—and a binary target column named Class. The data was explored, visualised, split into training and testing subsets using an 80:20 stratified split, and scaled with StandardScaler fitted only on the training data. The Logistic Regression model was trained on the scaled training set and evaluated on 275 held-out test records. At the default 0.50 threshold, it achieved 97.09% accuracy, 93.85% precision, 100% recall and a 96.83% F1-score for Class 1. A threshold experiment found that 0.70 reduced false positives from eight to one while retaining zero false negatives on this test split. These are experimental results on a public educational dataset and do not establish suitability for real banking decisions.

1. Introduction

Banknote authentication is a binary classification problem: each record is assigned to one of two class labels. In this project, the input measurements are numerical features extracted from scanned banknote images using a wavelet transform. Logistic Regression is used because it estimates the probability of a record belonging to a class and converts that probability into a class prediction using a decision threshold.

1.1 Problem statement

Build and evaluate a Logistic Regression model that uses four numerical measurements to predict the dataset's binary Class label. The model must be assessed on records not used during training, and its results must be interpreted using accuracy, precision, recall, F1-score, a confusion matrix and threshold analysis.

1.2 Objectives

- Understand the structure and quality of the Banknote Authentication dataset.
- Explore class distribution and visualise the input features.
- Prepare the input matrix and target variable and perform a stratified 80:20 split.
- Scale the features without leaking test-set information into training.
- Train a Logistic Regression classifier and generate class predictions and probabilities.
- Evaluate the classifier and investigate thresholds 0.30, 0.50 and 0.70.

2. Dataset Description and Exploration

The dataset was loaded from the UCI Machine Learning Repository. It has 1,372 records and five columns. The first four columns are continuous input features and Class is the integer target variable.

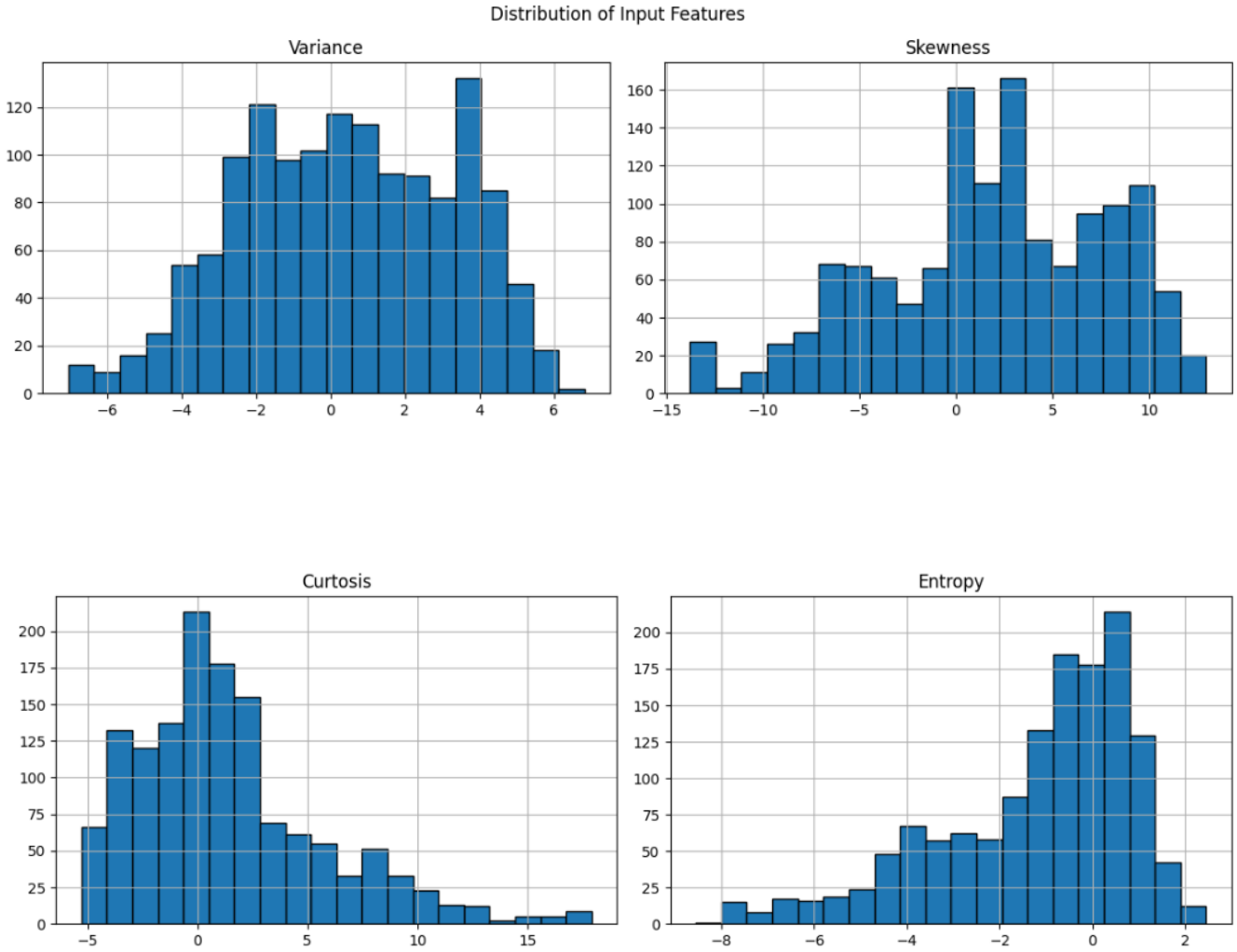
Column	Role	Description
Variance	Input feature	Variance of the wavelet-transformed image.
Skewness	Input feature	Skewness of the transformed image.
Curtosis	Input feature	Kurtosis-related measurement; the original dataset uses the name Curtosis.
Entropy	Input feature	Entropy of the image.
Class	Target	Binary class label, represented by 0 or 1.

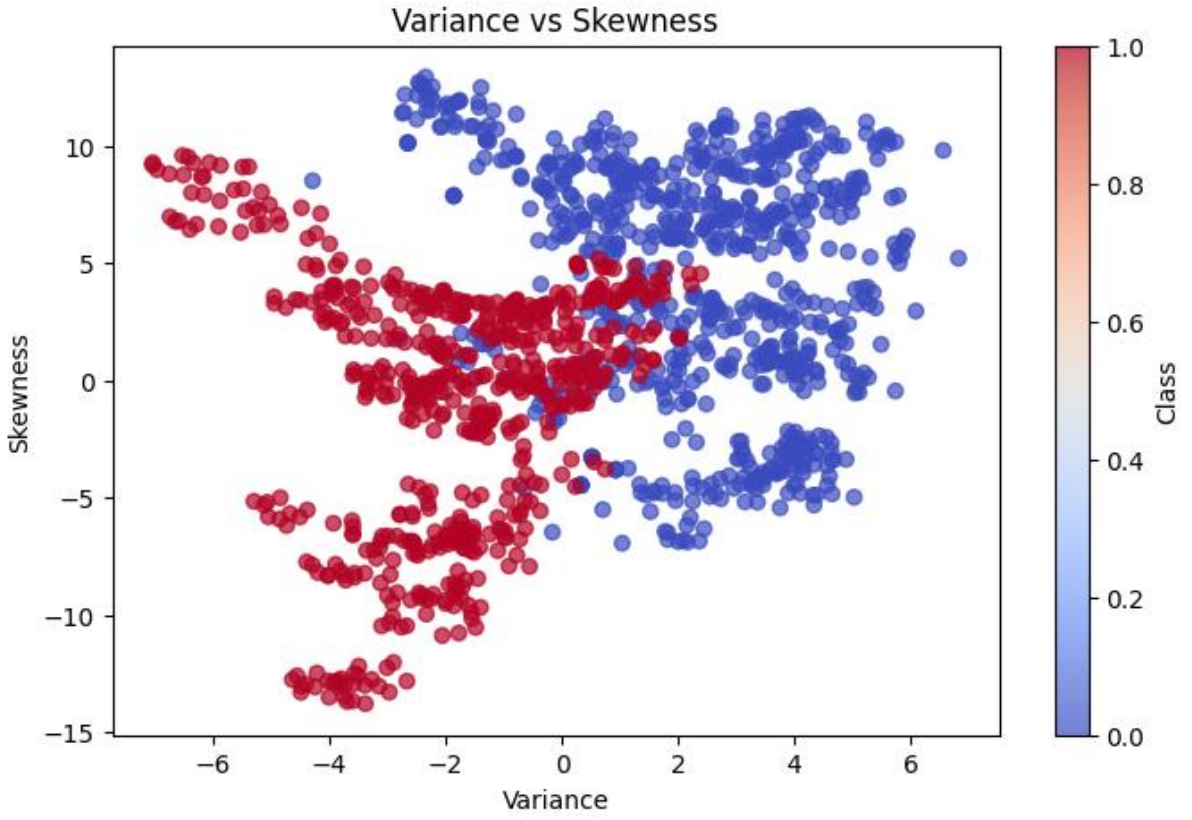
Exploration item	Observed result
Dataset dimensions	1,372 rows × 5 columns
Missing values	0 in every column
Duplicate records	24 duplicate rows detected; retained and reported, not removed.
Class 0	762 records (55.54%)
Class 1	610 records (44.46%)
Data types	Four float64 feature columns and one int64 target column.

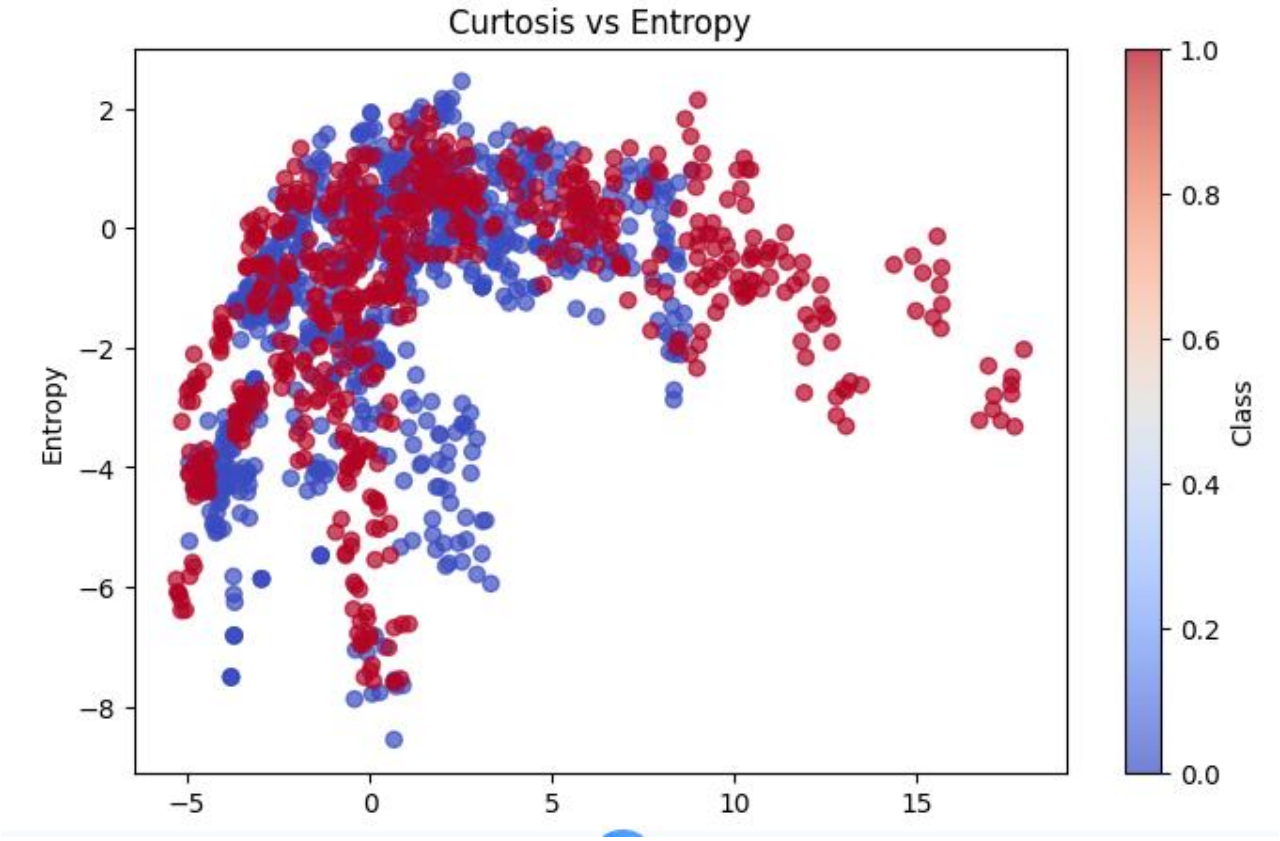
The class distribution is reasonably balanced: Class 0 represents 55.54% of records and Class 1 represents 44.46%. No missing values were detected. The feature summary statistics show that the input columns have different numerical ranges and standard deviations, supporting the use of feature scaling. The 24 duplicate records were reported but not automatically removed, consistent with the task instructions.

2.1 Visual exploration

The notebook includes a bar chart for class counts, four feature histograms and at least two feature-versus-feature scatter plots. Insert the original Colab histogram and scatter-plot screenshots in the marked evidence spaces in Appendix B. Add three observations based on the actual plots before submission; do not claim that any one feature guarantees correct classification




Variance vs Skewness
Skewness
Variance
Class
10
5
0
−5
−10
−15
−6
−4
−2
0
2
4
6
1.0
0.8
0.6
0.4
0.2
0.0


Curtosis vs Entropy
Entropy
Class
2
0
−2
−4
−6
−8
−5
0
5
10
15
1.0
0.8
0.6
0.4
0.2
0.0

3. Methodology

Stage	Implementation
Input and target	X = Variance, Skewness, Curtosis and Entropy; y = Class.
Train-test split	80% training (1,097 records) and 20% testing (275 records), random_state=42, stratify=y.
Feature scaling	StandardScaler fitted on X_train only; the fitted scaler transformed both X_train and X_test.
Model	Scikit-learn LogisticRegression(max_iter=1000).
Prediction	model.predict() generated class labels; model.predict_proba() generated probabilities for both classes.
Evaluation	Accuracy, precision, recall, F1-score, classification report and confusion matrix.
Threshold experiment	Class 1 predicted when its probability was at least 0.30, 0.50 or 0.70.

3.1 Why Logistic Regression?

Logistic Regression is suitable for a two-class classification task because it models the probability of a record belonging to a class. It uses the sigmoid function to map a model score to a value between 0 and 1. A threshold is then applied to turn the estimated probability into a predicted class.

3.2 Avoiding data leakage

The scaler was fitted only on the training data using fit_transform(). The same learned scaling parameters were then applied to the test data using transform(). This prevents information from the test set from influencing the preprocessing parameters used during training.

4. Results and Evaluation

The test set contained 275 records. The model correctly predicted 267 records and incorrectly predicted 8.

Metric (Class 1 positive)	Score	Interpretation
Accuracy	0.970909 (97.09%)	267 of 275 test predictions were correct.
Precision	0.938462 (93.85%)	Of the records predicted as Class 1, 93.85% were actually Class 1.
Recall	1.000000 (100%)	All actual Class 1 records in the test set were identified.
F1-score	0.968254 (96.83%)	Harmonic mean of Class 1 precision and recall.

4.1 Classification report

Class	Precision	Recall	F1-score	Support
0	1.00	0.95	0.97	153
1	0.94	1.00	0.97	122
Accuracy	—	—	0.97	275
Macro average	0.97	0.97	0.97	275
Weighted average	0.97	0.97	0.97	275

4.2 Confusion matrix at threshold 0.50

	Predicted Class 0	Predicted Class 1
Actual Class 0	TN = 145	FP = 8
Actual Class 1	FN = 0	TP = 122

The matrix is [[145, 8], [0, 122]] when labels are ordered [0, 1]. TN is an actual Class 0 record correctly predicted as 0; FP is an actual Class 0 record predicted as 1; FN is an actual Class 1 record predicted as 0; TP is an actual Class 1 record correctly predicted as 1. The positive class for the reported precision, recall and F1-score is Class 1.

The UCI dataset page describes the source specimens as genuine and forged, but its variable table labels the target simply as 'class' and does not explicitly map numeric 0 and 1 to genuine or forged. Therefore, this report retains the neutral Class 0 / Class 1 terminology rather than assigning an unsupported real-world meaning to the numeric labels. Confirm the convention with the course instructor or an authoritative dataset codebook before describing either class as genuine or forged.

5. Threshold Analysis

The same predicted test probabilities were converted into class labels using three thresholds. A record was predicted as Class 1 when its estimated probability for Class 1 was greater than or equal to the selected threshold.

Threshold	TN	FP	FN	TP	Accuracy	Precision	Recall	F1-score
0.30	145	8	0	122	97.09%	93.85%	100.00%	96.83%
0.50	145	8	0	122	97.09%	93.85%	100.00%	96.83%
0.70	152	1	0	122	99.64%	99.19%	100.00%	99.59%

Thresholds 0.30 and 0.50 produced identical predictions on this test set. At 0.70, the number of false positives fell from 8 to 1, while false negatives remained at 0. The corresponding accuracy rose to 99.64%, precision to 99.19%, and F1-score to 99.59%. Among the three thresholds tested, 0.70 performed best on this particular test split. This does not prove that 0.70 is universally optimal: threshold selection should ideally be performed on a separate validation set, with final performance measured once on an untouched test set.

The costs of false positives and false negatives depend on the real application. If failing to flag a forged note is especially costly, recall for the class representing forged notes would be important. Because the official UCI variable description does not explicitly map the numeric labels, the class-to-forgery mapping must be verified before making that application-specific claim.

6. Answers to Key Record Questions

1. Regression vs classification

Regression predicts a continuous numerical value; classification predicts a category or class label.

2. Why is Logistic Regression used for classification?

Despite its name, it estimates class probabilities using a logistic (sigmoid) function and assigns a class using a threshold.

3. What are the four input features?

Variance, Skewness, Curtosis and Entropy.

4. What is the target variable?

Class, represented by 0 or 1. The UCI variable table does not explicitly map these numbers to genuine/forged labels.

5. How many records and features are present?

1,372 records, four input features and one target column; no missing values and 24 duplicate rows detected.

6. Is the class distribution balanced?

Reasonably balanced: Class 0 = 762 (55.54%); Class 1 = 610 (44.46%).

7. Why split into training and testing subsets?

To train the model on one subset and evaluate it on unseen records.

8. What is stratified splitting?

It preserves approximately the same class proportions in the training and test subsets.

9. Why scale features?

Scaling places features with different numerical ranges on comparable scales, which is useful for Logistic Regression optimisation.

10. What is data leakage?

It occurs when information from the test set influences training or preprocessing; fitting the scaler before splitting can leak test-set distribution information.

11. What does the sigmoid function do?

It maps a model score to a value between 0 and 1 that can be interpreted as an estimated class probability.

12. What does probability 0.80 for Class 1 mean?

The model estimates an 80% probability for Class 1; it is not a guarantee that the prediction is correct.

13. Explain the evaluation metrics.

Accuracy is the overall fraction correct; precision is the fraction of predicted positives that are truly positive; recall is the fraction of actual positives detected; F1 is the harmonic mean of precision and recall.

14. Explain TP, TN, FP and FN.

TP: actual 1 predicted 1; TN: actual 0 predicted 0; FP: actual 0 predicted 1; FN: actual 1 predicted 0.

15. How did thresholds affect results?

0.30 and 0.50 were identical. At 0.70, FP decreased from 8 to 1, with FN remaining 0; accuracy and precision increased on this test split.

16. Which metric matters if missing a forged note is costly?

Recall for the forged-note class would be important, but first verify which numeric label represents forged notes.

17. State two limitations.

The dataset is a fixed public dataset with a limited set of numerical features; results on one random split may not generalise to new currencies, imaging conditions or real-world banknote screening.

7. Conclusion

This project implemented a Logistic Regression classifier for the UCI Banknote Authentication dataset using four numerical features: Variance, Skewness, Curtosis and Entropy. The dataset contained 1,372 records and five columns. Initial exploration found no missing values and identified 24 duplicate records, which were reported and retained. The data was divided using an 80:20 stratified split, producing 1,097 training records and 275 testing records. StandardScaler was fitted only on the training data to avoid data leakage. The trained model achieved 97.09% accuracy, 93.85% precision, 100% recall and a 96.83% F1-score for Class 1 at the default 0.50 threshold. Threshold analysis showed that 0.30 and 0.50 produced the same confusion matrix, while 0.70 reduced false positives from eight to one and achieved 99.64% accuracy on this test split. The results show that threshold selection can change the balance of classification errors. For an application where missing a forged note is costly, recall for the forged-note class would deserve particular attention, after confirming the numeric class mapping. This study is limited by its fixed dataset, selected features and single train-test split. It is an educational experiment and is not a certified banknote-authentication system.

8. References

1. Lohweg, V. (2012). Banknote Authentication [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55P57

2. UCI Machine Learning Repository. Banknote Authentication. https://archive.ics.uci.edu/dataset/267/banknote+authentication

3. Scikit-learn documentation. LogisticRegression, StandardScaler, train_test_split and classification metrics. https://scikit-learn.org/

4. Machine Learning Lab, Department of CSE – AI & ML, The National Institute of Engineering, Mysuru. Banknote Authentication using Logistic Regression: Student Project Instructions.

Appendix A – Colab screenshots already provided

The following screenshots were attached in the project conversation and have been inserted as evidence. Some screenshots show code cells and their outputs; keep them as supporting notebook evidence.

Evidence A1: Library imports and setup

```
#imported libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
#dataset loading
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00267/data_banknote_authentication.txt"

columns = [
    "Variance",
    "Skewness",
    "Curtosis",
```

Figure A1. Library imports and setup.

Evidence A2: Dataset loading and dataset dimensions

```
All libraries imported successfully!
```

```
#dataset loading
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00267/data_banknote_authentication.txt"

columns = [
    "Variance",
    "Skewness",
    "Curtosis",
    "Entropy",
    "Class"
]

df = pd.read_csv(url, header=None, names=columns)

print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

```
df.head(10)
print("Dataset shape:", df.shape)
```

```
Dataset shape: (1372, 5)
```

Figure A2. Dataset loading and dataset dimensions.

Evidence A3: DataFrame information and data types

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1372 entries, 0 to 1371
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   Variance  1372 non-null   float64
 1   Skewness  1372 non-null   float64
 2   Curtosis  1372 non-null   float64
 3   Entropy   1372 non-null   float64
 4   Class     1372 non-null   int64  
dtypes: float64(4), int64(1)
memory usage: 53.7 KB
```

```
print("Missing values in each column:")
print(df.isnull().sum())
print("Number of duplicate records:", df.duplicated().sum())
```

```
Missing values in each column:
Variance    0
Skewness    0
Curtosis    0
Entropy     0
Class       0
dtype: int64
Number of duplicate records: 24
```

Figure A3. DataFrame information and data types.

Evidence A4: Descriptive statistics

```
df.describe()
```

	Variance	Skewness	Curtosis	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Figure A4. Descriptive statistics.

Evidence A5: Class counts and class percentages

```
#distribustion of class
print("Class counts:")
print(df["Class"].value_counts())

print("\nClass percentages:")
print(df["Class"].value_counts(normalize=True) * 100)
```

```
Class counts:
Class
0    762
1    610
Name: count, dtype: int64

Class percentages:
Class
0    55.539359
1    44.460641
Name: proportion, dtype: float64
```

Figure A5. Class counts and class percentages.

Evidence A6: Class distribution plotting code

```
#graph
df["Class"].value_counts().sort_index().plot(
    kind="bar",
    color=["skyblue", "orange"],
    edgecolor="black"
)

plt.title("Distribution of Banknote Classes")
plt.xlabel("Class")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.show()
```

Figure A6. Class distribution plotting code.

Evidence A7: Class distribution bar chart

```
plt.title("Distribution of Banknote Classes")
plt.xlabel("Class")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.show()
```

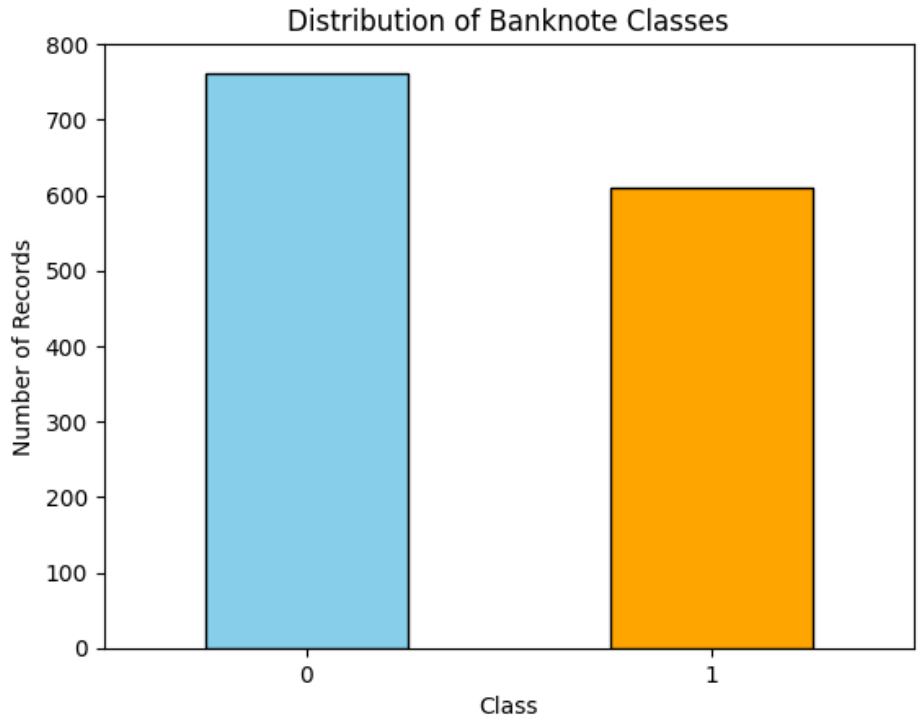


Figure A7. Class distribution bar chart.

Evidence A8: Train-test split and StandardScaler outputs

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

print("Training labels:", y_train.shape)
print("Testing labels:", y_test.shape)
```

```
Training data: (1097, 4)
Testing data: (275, 4)
Training labels: (1097,)
Testing labels: (275,)
```

```
#stage 6
#Apply StandardScaler
# Feature scaling

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed successfully!")

print("Scaled training data shape:", X_train_scaled.shape)
print("Scaled testing data shape:", X_test_scaled.shape)
```

```
Feature scaling completed successfully!
Scaled training data shape: (1097, 4)
Scaled testing data shape: (275, 4)
```

Figure A8. Train-test split and StandardScaler outputs.

Evidence A9: Logistic Regression model training

```
#stage 7
# Creating the Logistic Regression model

model = LogisticRegression(max_iter=1000)

# Training the model

model.fit(X_train_scaled, y_train)

print("Logistic Regression model trained successfully!")
```

```
Logistic Regression model trained successfully!
```

Figure A9. Logistic Regression model training.

Evidence A10: Predictions and probability generation

```
#stage 8
#Predicting the test data

y_pred = model.predict(X_test_scaled)

print("Predictions generated successfully!")
print("Total predictions:", len(y_pred))


#Comparing actual and predicted values

comparison = pd.DataFrame({
    "Actual Class": y_test.values,
    "Predicted Class": y_pred
})

comparison.head(15)
# Generate prediction probabilities

y_prob = model.predict_proba(X_test_scaled)

print("Probability predictions generated successfully!")
print(y_prob[:5])
```

Figure A10. Predictions and probability generation.

Evidence A11: Probability table and correct/incorrect prediction counts

```
#Displaying actual, predicted and probability values
probability_results = pd.DataFrame({
    "Actual Class": y_test.values,
    "Predicted Class": y_pred,
    "Probability of Class 0": y_prob[:, 0],
    "Probability of Class 1": y_prob[:, 1]
})

probability_results.head(10)

#Checking correct and incorrect predictions

correct = (y_test.values == y_pred).sum()
incorrect = (y_test.values != y_pred).sum()

print("Correct predictions:", correct)
print("Incorrect predictions:", incorrect)
print("Total predictions:", len(y_pred))
```

```
Predictions generated successfully!
Total predictions: 275
Probability predictions generated successfully!
[[9.99576433e-01 4.23567092e-04]
 [3.35717098e-02 9.66428290e-01]
 [7.55630532e-01 2.44369468e-01]
 [3.55676247e-02 9.64432375e-01]
 [6.95876485e-03 9.93041235e-01]]
Correct predictions: 267
Incorrect predictions: 8
Total predictions: 275
```

Figure A11. Probability table and correct/incorrect prediction counts.

Evidence A12: Model evaluation code

```
#stage 9
# Model Evaluation

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred)

recall = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)

print("Model Evaluation Results")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

#Complete Classification Report
print("Classification Report:\n")
print(classification_report(y_test, y_pred))

#Evaluation Metrics Table
evaluation_results = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
    "Score": [accuracy, precision, recall, f1]
})

evaluation_results
```

Figure A12. Model evaluation code.

Evidence A13: Classification report and evaluation metrics

```
Model Evaluation Results
Accuracy: 0.9709090909090909
Precision: 0.9384615384615385
Recall: 1.0
F1 Score: 0.9682539682539683
Classification Report:
```

	precision	recall	f1-score	support
0	1.00	0.95	0.97	153
1	0.94	1.00	0.97	122
accuracy			0.97	275
macro avg	0.97	0.97	0.97	275
weighted avg	0.97	0.97	0.97	275

	Metric	Score
0	Accuracy	0.970909
1	Precision	0.938462
2	Recall	1.000000
3	F1-Score	0.968254

Figure A13. Classification report and evaluation metrics.

Evidence A14: Confusion matrix code

```
#stage 10
# Generating Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

#Extracting confusion matrix values
TN, FP, FN, TP = cm.ravel()
print("True Negative (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)
print("True Positive (TP):", TP)

#Visualising the Confusion Matrix
plt.figure(figsize=(7, 5))

plt.imshow(cm, interpolation="nearest", cmap="Blues")

plt.title("Confusion Matrix - Logistic Regression")
plt.colorbar()

plt.xticks([0, 1], ["Class 0", "Class 1"])
plt.yticks([0, 1], ["Class 0", "Class 1"])

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
```

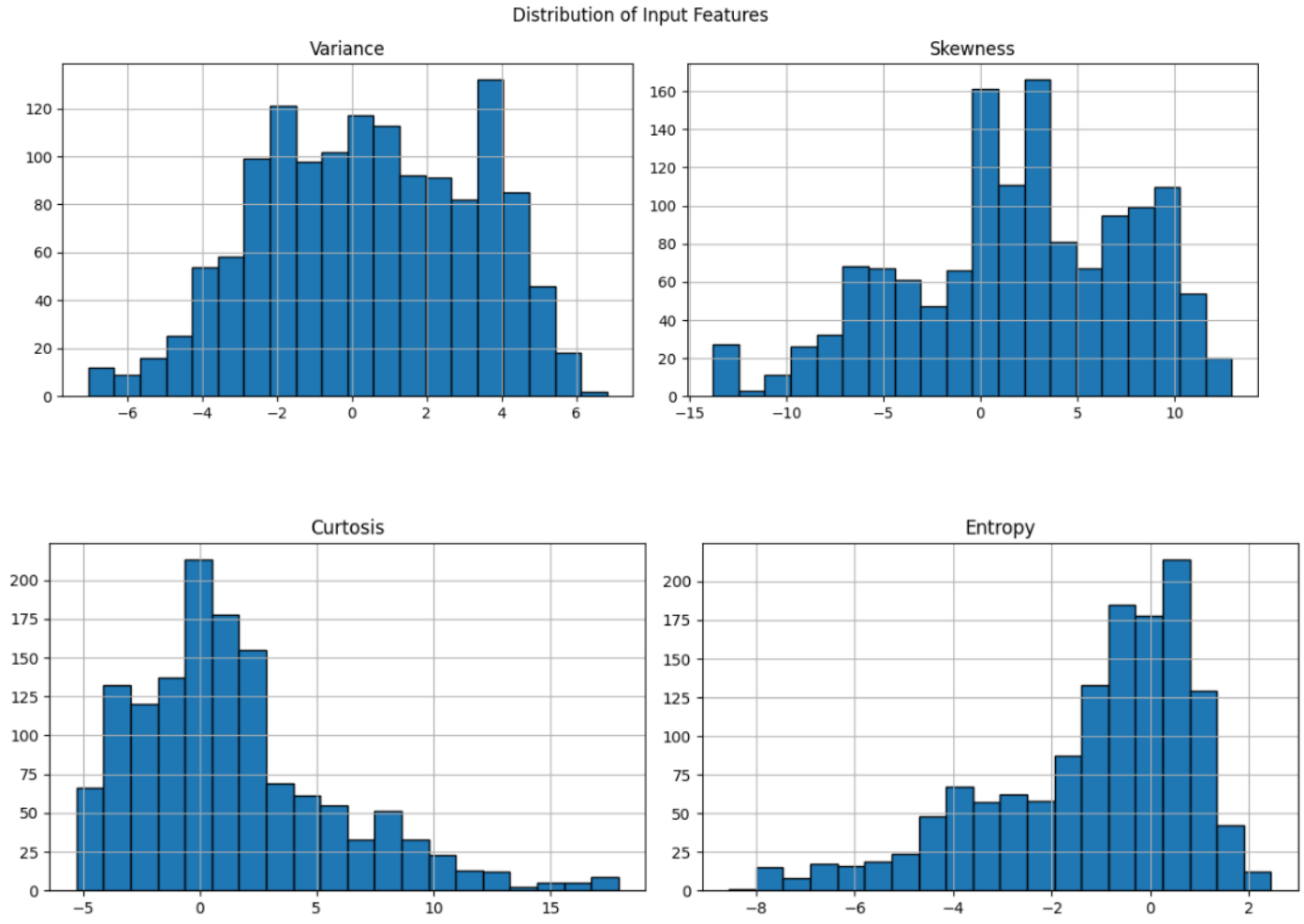
Figure A14. Confusion matrix code.

Appendix B – Add the remaining screenshots

The following screenshot spaces are intentionally left for figures that were not attached in the conversation. In Word, click the placeholder text and use Insert → Pictures to add the corresponding image exported from Colab.

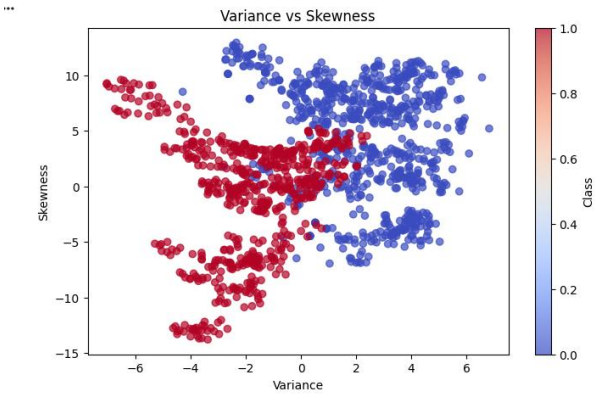
B1. Four feature histograms

Insert the histogram output showing Variance, Skewness, Curtosis and Entropy.



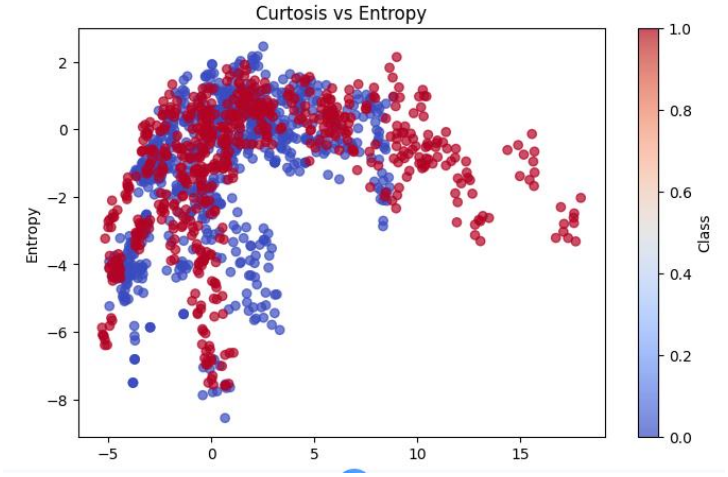
B2. Feature scatter plot 1

Ins**ert** the Variance versus Skewness scatter plot with class colours.



B3. Feature scatter plot 2

Insert the Curtosis versus Entropy scatter plot with class colours.



B5. Probability results

Insert the table showing actual class, predicted class and both class probabilities.

```
#Displaying actual, predicted and probability values
probability_results = pd.DataFrame({
    "Actual Class": y_test.values,
    "Predicted Class": y_pred,
    "Probability of Class 0": y_prob[:, 0],
    "Probability of Class 1": y_prob[:, 1]
})

probability_results.head(10)

#Checking correct and incorrect predictions

correct = (y_test.values == y_pred).sum()
incorrect = (y_test.values != y_pred).sum()

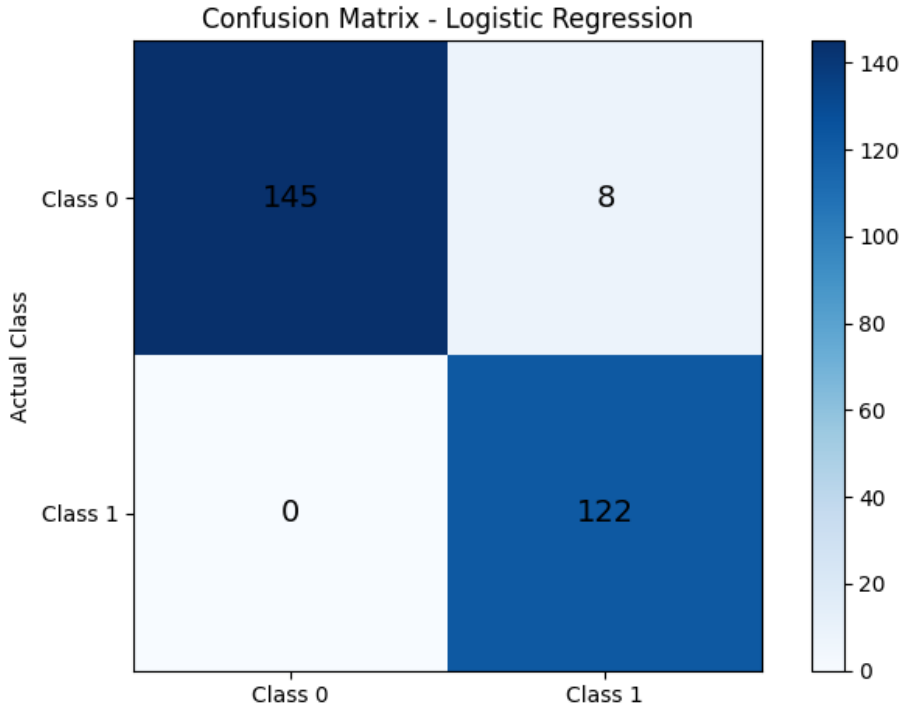
print("Correct predictions:", correct)
print("Incorrect predictions:", incorrect)
print("Total predictions:", len(y_pred))
```

```
Predictions generated successfully!
Total predictions: 275
Probability predictions generated successfully!
[[9.99576433e-01 4.23567092e-04]
 [3.35717098e-02 9.66428290e-01]
 [7.55630532e-01 2.44369468e-01]
 [3.55676247e-02 9.64432375e-01]
 [6.95876485e-03 9.93041235e-01]]
Correct predictions: 267
Incorrect predictions: 8
Total predictions: 275
```

B6. Confusion matrix output and heatmap

Insert the numerical matrix, TN/FP/FN/TP values and labelled heatmap.

```
Confusion Matrix:
[[145   8]
 [  0 122]]
True Negative (TN): 145
False Positive (FP): 8
False Negative (FN): 0
True Positive (TP): 122
```



B7. Threshold comparison table

Insert the Colab table comparing accuracy, precision, recall and F1-score at 0.30, 0.50 and 0.70.

```
Threshold analysis completed!
 Threshold  Accuracy  Precision  Recall  F1-Score     Confusion Matrix
       0.3  0.970909   0.938462     1.0  0.968254 [[145, 8], [0, 122]]
       0.5  0.970909   0.938462     1.0  0.968254 [[145, 8], [0, 122]]
       0.7  0.996364   0.991870     1.0  0.995918 [[152, 1], [0, 122]]
```

B8. Threshold confusion matrices

Insert the three confusion matrices for thresholds 0.30, 0.50 and 0.70.

```
Threshold analysis completed!

Threshold: 0.3
[[145   8]
 [  0 122]]

Threshold: 0.5
[[145   8]
 [  0 122]]

Threshold: 0.7
[[152   1]
 [  0 122]]
```